

```
# data sources:https://www.kaggle.com/datasets/adarshsng/lending-club-loan-data-csv
```

Double-click (or enter) to edit

```
from google.colab import files
files.upload()
```



选择文件 kaggle.json

- **kaggle.json**(application/json) – 74 bytes, last modified: 2025/6/20 – 100% done
- Saving kaggle.json to kaggle.json

```
import os
os.makedirs("/root/.kaggle", exist_ok=True)
!mv kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json
```

```
!kaggle datasets download -d adarshsng/lending-club-loan-data-csv
```



Dataset URL: <https://www.kaggle.com/datasets/adarshsng/lending-club-loan-data-csv>
 License(s): DbCL-1.0
 Downloading lending-club-loan-data-csv.zip to /content
 96% 326M/339M [00:01<00:00, 249MB/s]
 100% 339M/339M [00:01<00:00, 295MB/s]

```
import zipfile
with zipfile.ZipFile("/content/lending-club-loan-data-csv.zip","r") as Lending_club:
    Lending_club.extractall("lending-club-loan-data-csv")
```

```
import zipfile
with zipfile.ZipFile("/content/lending-club-loan-data-csv.zip","r") as Lending_club:
    Lending_club.extractall("lending-club-loan-data-csv")
```

```
from google.colab import drive
drive.mount("/content/drive")
```



Mounted at /content/drive

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from tabulate import tabulate
```

```
lending_club = pd.read_csv("/content/lending-club-loan-data-csv/loan.csv")
lending_club.head()
```

```
↳ /tmp/ipython-input-9-970121203.py:1: DtypeWarning: Columns (19,47,55,112,123,124,125,128,129,130,133,139,140,141) h
lending_club = pd.read_csv("/content/lending-club-loan-data-csv/loan.csv")
```

	id	member_id	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	sub_grade	...	hard:
0	NaN	NaN	2500	2500	2500.0	36 months	13.56	84.92	C	C1	...	
1	NaN	NaN	30000	30000	30000.0	60 months	18.94	777.23	D	D2	...	
2	NaN	NaN	5000	5000	5000.0	36 months	17.97	180.69	D	D1	...	
3	NaN	NaN	4000	4000	4000.0	36 months	18.94	146.51	D	D2	...	
4	NaN	NaN	30000	30000	30000.0	60 months	16.14	731.78	C	C4	...	

5 rows × 145 columns

```
lending_club.isnull().sum()
```

	0
id	2260668
member_id	2260668
loan_amnt	0
funded_amnt	0
funded_amnt_inv	0
...	...
settlement_status	2227612
settlement_date	2227612
settlement_amount	2227612
settlement_percentage	2227612
settlement_term	2227612

145 rows × 1 columns

```
thresh = len(lending_club) * 0.7
lending_club = lending_club.dropna(thresh=thresh, axis=1)
lending_club = lending_club.dropna()
lending_club.head()
```

	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	sub_grade	emp_title	emp_length
0	2500	2500	2500.0	36 months	13.56	84.92	C	C1	Chef	10+ years
1	30000	30000	30000.0	60 months	18.94	777.23	D	D2	Postmaster	10+ years
2	5000	5000	5000.0	36 months	17.97	180.69	D	D1	Administrative	6 years
3	4000	4000	4000.0	36 months	18.94	146.51	D	D2	IT Supervisor	10+ years
4	30000	30000	30000.0	60 months	16.14	731.78	C	C4	Mechanic	10+ years

5 rows × 87 columns

```
lending_club.nunique()
```



	0
loan_amnt	1561
funded_amnt	1561
funded_amnt_inv	1603
term	2
int_rate	356
...	...
total_bc_limit	9777
total_il_high_credit_limit	186578
hardship_flag	2
disbursement_method	2
debt_settlement_flag	2

87 rows × 1 columns

```
lending_club['loan_status'].value_counts()
```



	count
Fully Paid	764818
Current	671801
Charged Off	190439
Late (31-120 days)	15965
In Grace Period	6768
Late (16-30 days)	2635
Default	25

```
import xgboost as xgb
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

```
lending_club = lending_club.copy()
y=lending_club["loan_status"]
x= lending_club.drop(columns = ["loan_status"])
le = LabelEncoder()
```

```
for col in x.select_dtypes(include='object').columns:
    x[col] = LabelEncoder().fit_transform(x[col].astype(str))
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)
model = xgb.XGBClassifier(max_depth=6,n_estimators = 100,use_label_encoder=False,eval_metric = "logloss")
model.fit(x_train, y_train_encoded)
```

```

/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [06:37:43] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.

```

```
warnings.warn(msg, UserWarning)
```

```

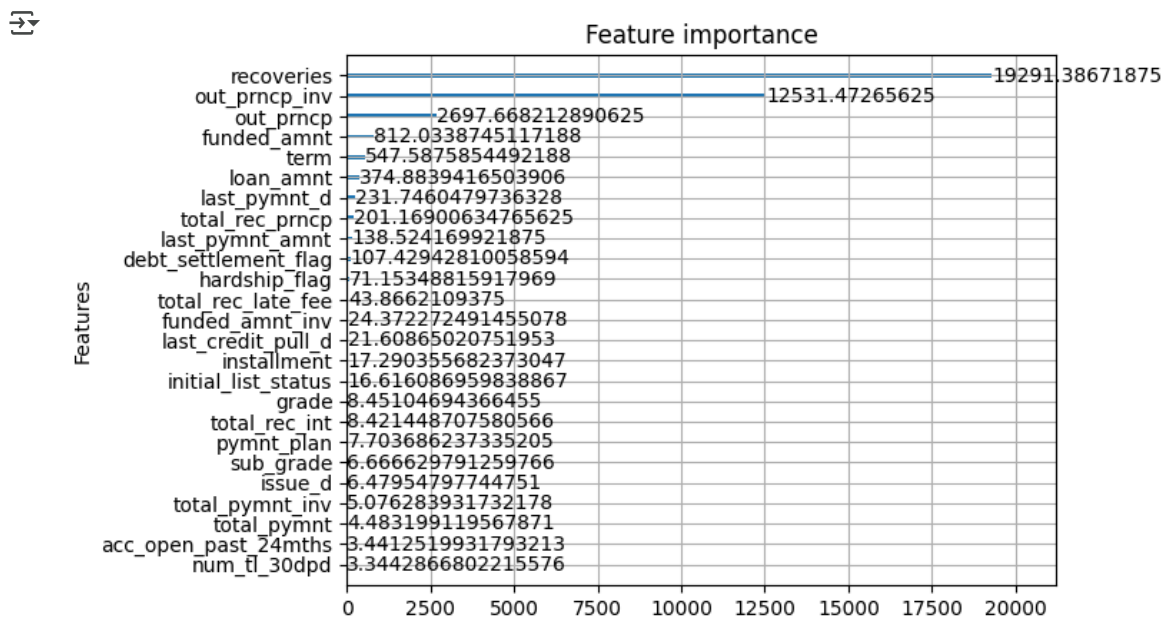
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='logloss',
               feature_types=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=None, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=100,
               n_jobs=None, num_parallel_tree=None, objective='multi:softprob', ...)

```

```

import matplotlib.pyplot as plt
xgb.plot_importance(model, max_num_features=25, importance_type='gain')
plt.show()

```



```

from sklearn.feature_selection import RFE
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)

gain_dict = model.get_booster().get_score(importance_type='gain')
sorted_features = sorted(gain_dict.items(), key=lambda x: x[1], reverse=True)
top_15 = [f[0] for f in sorted_features[:15]]
x_train_top15 = x_train[top_15]

rfe = RFE(estimator=model, n_features_to_select=10, step=1)
rfe.fit(x_train_top15, y_train_encoded)

selected_10 = x_train_top15.columns[rfe.support_]
print("The most important 10 features", selected_10.tolist())

```

```

/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [06:42:17] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.

```

```

warnings.warn(msg, UserWarning)
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [06:44:04] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.

```

```
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [06:45:49] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.
```

```
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [06:47:37] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.
```

```
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [06:49:18] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.
```

```
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [06:50:57] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.
```

```
warnings.warn(msg, UserWarning)
The most important 10 features ['recoveries', 'out_prncp_inv', 'out_prncp', 'funded_amnt', 'term', 'last_pymnt_d',
```

```
import pandas as pd
print(pd.Series(y_train).value_counts())
```

```
↗ loan_status
Fully Paid      611631
Current         537982
Charged Off     152047
Late (31-120 days)  12788
In Grace Period   5354
Late (16-30 days)  2139
Default          19
Name: count, dtype: int64
```

```
import numpy as np
unique_classes, counts = np.unique(y_train_encoded, return_counts=True)
print("Category and Sample size: ")
for cls, cnt in zip(unique_classes, counts):
    print(f"Category {cls}: {cnt}")
```

```
↗ Category and Sample size:
Category 0: 152047
Category 1: 537982
Category 2: 19
Category 3: 611631
Category 4: 5354
Category 5: 2139
Category 6: 12788
```

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.set(style="whitegrid")
selected_features = [
    'recoveries', 'out_prncp_inv', 'out_prncp', 'funded_amnt', 'term',
    'last_pymnt_d', 'total_rec_prncp', 'last_pymnt_amnt',
    'debt_settlement_flag', 'hardship_flag'
]
df_selected = lending_club[selected_features].copy()
df_selected['term'] = df_selected['term'].str.extract('(\d+)').astype(float)

df_selected['last_pymnt_d'] = pd.to_datetime(df_selected['last_pymnt_d'], format='%b-%Y', errors='coerce')
df_selected['last_pymnt_d'] = df_selected['last_pymnt_d'].map(lambda x: x.toordinal() if pd.notnull(x) else np.nan)

df_selected['debt_settlement_flag'] = df_selected['debt_settlement_flag'].map({'Y': 1, 'N': 0})
df_selected['hardship_flag'] = df_selected['hardship_flag'].map({'Y': 1, 'N': 0})

corr_matrix = df_selected.corr()

sns.set()
plt.figure(figsize=(8, 6))
```

```

ax = sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    vmin=-1, vmax=1, center=0,
    cmap="YlOrBr",
    square=True,
    linewidths=0.5,
    cbar_kws={"shrink": 0.75}
)

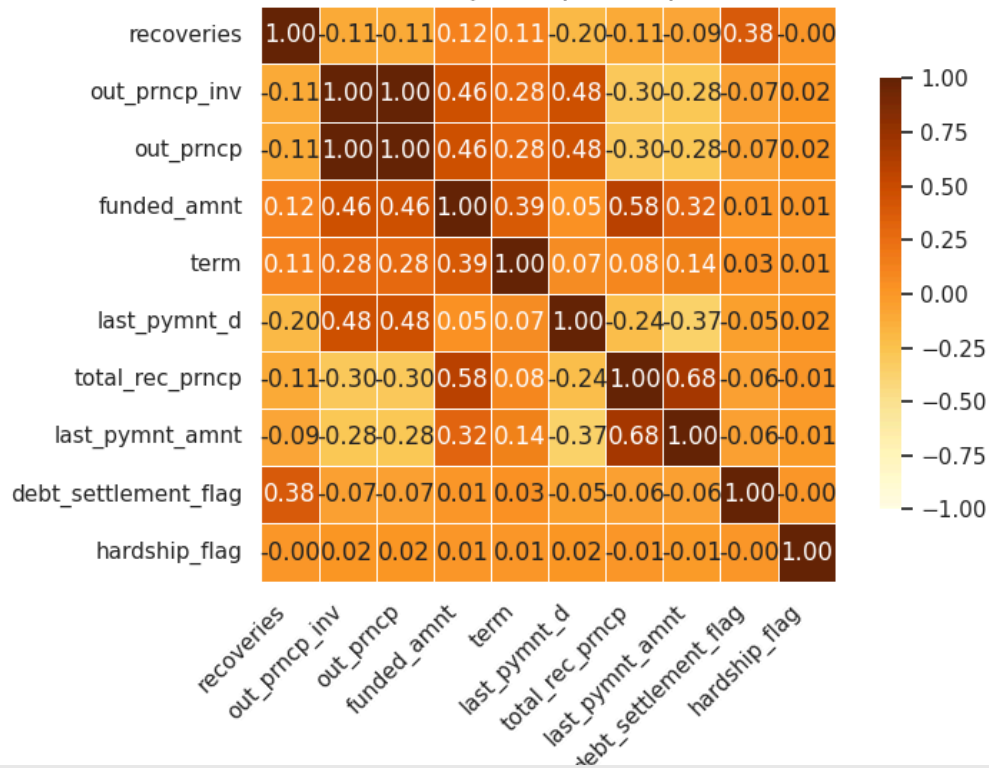
ax.set_xticklabels(
    ax.get_xticklabels(),
    rotation=45,
    horizontalalignment='right'
)

plt.title("Correlation Heatmap of Top 10 Important Features", fontsize=14)
plt.tight_layout()
plt.show()

```



Correlation Heatmap of Top 10 Important Features



```

corr_matrix = df_selected.corr()
print(corr_matrix)

```



```

recoveries    recoveries    out_prncp_inv    out_prncp    funded_amnt    \
recoveries    1.000000    -0.113449    -0.113459    0.122203
out_prncp_inv -0.113449    1.000000    0.999999    0.462480
out_prncp     -0.113459    0.999999    1.000000    0.462468
funded_amnt   0.122203    0.462480    0.462468    1.000000
term          0.111214    0.276742    0.276664    0.390726
last_pymnt_d  -0.195096    0.475785    0.475830    0.052388
total_rec_prncp -0.112055    -0.304075    -0.304080    0.575837
last_pymnt_amnt -0.092060    -0.282501    -0.282528    0.317618
debt_settlement_flag 0.380407    -0.069136    -0.069142    0.006030
hardship_flag -0.003819    0.020557    0.020562    0.009334

term    last_pymnt_d    total_rec_prncp    \
recoveries    0.111214    -0.195096    -0.112055
out_prncp_inv 0.276742    0.475785    -0.304075
out_prncp     0.276664    0.475830    -0.304080

```

funded_amnt	0.390726	0.052388	0.575837
term	1.000000	0.073020	0.083611
last_pymnt_d	0.073020	1.000000	-0.241275
total_rec_prncp	0.083611	-0.241275	1.000000
last_pymnt_amnt	0.139689	-0.366516	0.678772
debt_settlement_flag	0.027055	-0.050212	-0.055582
hardship_flag	0.006637	0.015854	-0.005236

	last_pymnt_amnt	debt_settlement_flag	hardship_flag
recoveries	-0.092060	0.380407	-0.003819
out_prncp_inv	-0.282501	-0.069136	0.020557
out_prncp	-0.282528	-0.069142	0.020562
funded_amnt	0.317618	0.006030	0.009334
term	0.139689	0.027055	0.006637
last_pymnt_d	-0.366516	-0.050212	0.015854
total_rec_prncp	0.678772	-0.055582	-0.005236
last_pymnt_amnt	1.000000	-0.062363	-0.010733
debt_settlement_flag	-0.062363	1.000000	-0.002457
hardship_flag	-0.010733	-0.002457	1.000000

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
loan_status_counts = pd.Series({
    'Fully Paid': 611631,
    'Current': 537982,
    'Charged Off': 152047,
    'Late (31-120 days)': 12788,
    'In Grace Period': 5354,
    'Late (16-30 days)': 2139,
    'Default': 19
})
```

```
df_plot = loan_status_counts.reset_index()
df_plot.columns = ['loan_status', 'count']
```

```
sns.set(style="whitegrid")
```

```
plt.figure(figsize=(8, 6))
ax = sns.barplot(
    data=df_plot,
    x='loan_status',
    y='count',
    color='#4c72b0',
    width=0.5
)
```

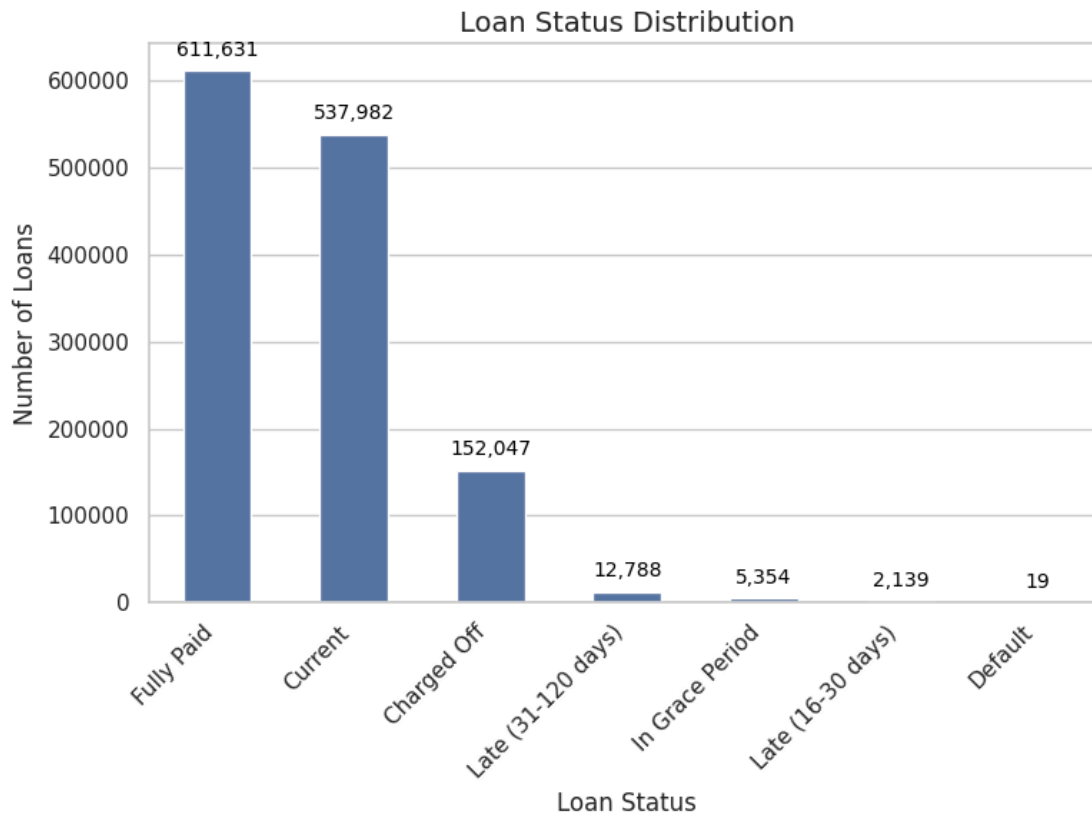
```
for index, row in df_plot.iterrows():
    ax.text(
        index, row['count'] + max(df_plot['count']) * 0.02,
        f"{row['count']:,}",
        color='black',
        ha='center',
        va='bottom',
        fontsize=10
    )
```

```
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right')
```

```
plt.title("Loan Status Distribution", fontsize=14)
plt.xlabel("Loan Status")
plt.ylabel("Number of Loans")
```

```
plt.tight_layout()
plt.show()
```

 /tmp/ipython-input-33-3794167459.py:45: UserWarning: set_ticklabels() should only be used with a fixed number of ticks.
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right')



```
#Cold Start
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df_cold = lending_club[lending_club['loan_status'].isin(['Current', 'Issued'])].copy()

features = ['recoveries', 'out_prncp_inv', 'out_prncp', 'funded_amnt', 'term',
            'total_rec_prncp', 'last_pymnt_amnt', 'debt_settlement_flag', 'hardship_flag']

valid_features = [col for col in features if (pd.to_numeric(df_cold[col], errors='coerce').isna().mean() <= 0.7)]
df_cold[valid_features] = df_cold[valid_features].apply(lambda col: pd.to_numeric(col, errors='coerce').fillna(0))

if 'term' in valid_features:
    df_cold['term'] = df_cold['term'].astype(str).str.extract('(\d+)').astype(int)
if 'debt_settlement_flag' in valid_features:
    df_cold['debt_settlement_flag'] = df_cold['debt_settlement_flag'].map({'Y':1, 'N':0})
if 'hardship_flag' in valid_features:
    df_cold['hardship_flag'] = df_cold['hardship_flag'].map({'Y':1, 'N':0})

x = df_cold[valid_features]

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(x_scaled)
df_cold['cluster'] = clusters

print(df_cold.groupby('cluster')[valid_features].mean())
print(df_cold['cluster'].value_counts())
```

 cluster recoveries out_prncp_inv out_prncp funded_amnt \

cluster	recoveries	out_prncp_inv	out_prncp	funded_amnt
0	0.0	6489.451914	6491.512143	9912.061615


```

1          0.0    7962.203129    7965.453597    24397.695461
2          0.0    23125.629793    23128.235330    27885.372329

```

```

total_rec_prncp  last_pymnt_amnt
cluster
0          3420.549472          304.681760
1         16432.241864          961.135575
2          4757.136998          766.042685

```

```

cluster
0    421519
2    161121
1     89161
Name: count, dtype: int64

```

```
from sklearn.decomposition import PCA
```

```
# 降到2维
```

```
pca = PCA(n_components=2, random_state=42)
```

```
x_pca = pca.fit_transform(x_scaled)
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8,6))
```

```
scatter = plt.scatter(x_pca[:,0], x_pca[:,1], c=clusters, cmap='viridis', alpha=0.6)
```

```
plt.xlabel('PCA 1')
```

```
plt.ylabel('PCA 2')
```

```
plt.title('KMeans Clustering Visualization (PCA 2D)')
```

```
plt.colorbar(scatter, label='Cluster')
```

```
plt.show()
```

